

Transport Layer Security / TLS

Standard for Internet security

- Goal: “... provide privacy and reliability between two communicating applications”

Two main parts

1. Handshake Protocol: **Establish shared secret key using public-key cryptography**
2. Record Layer: **Transmit data using negotiated key**

Our starting point: Using a key for encryption and integrity

AES-NI: AES in hardware (Intel, AMD, ARM)

New x86 hardware instructions used to implement AES:

- **aesenc, aesenclast**: one round of AES

aesenc **xmm1**, **xmm2** (result written to xmm1)

state round key

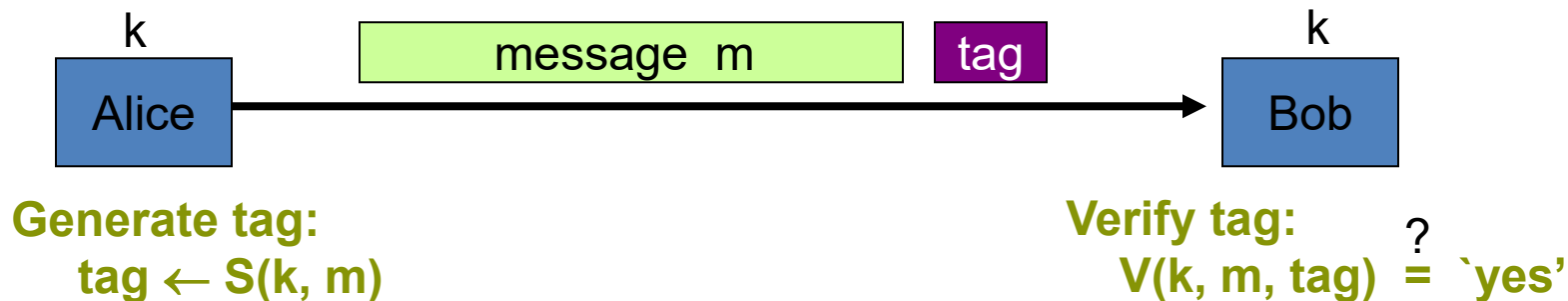
- **aesdec, aesdeclast**: one round of AES
- **aeskeygenassist**: do AES key expansion

⇒ more than 10x speedup over a software AES

⇒ better security: all AES instructions are **constant time**

Message Integrity: MACs

- Goal: provide message integrity. No confidentiality.
 - ex: Protecting public binaries on disk.



Construction: HMAC (Hash-MAC)

Most widely used MAC on the Internet.

H: hash function.

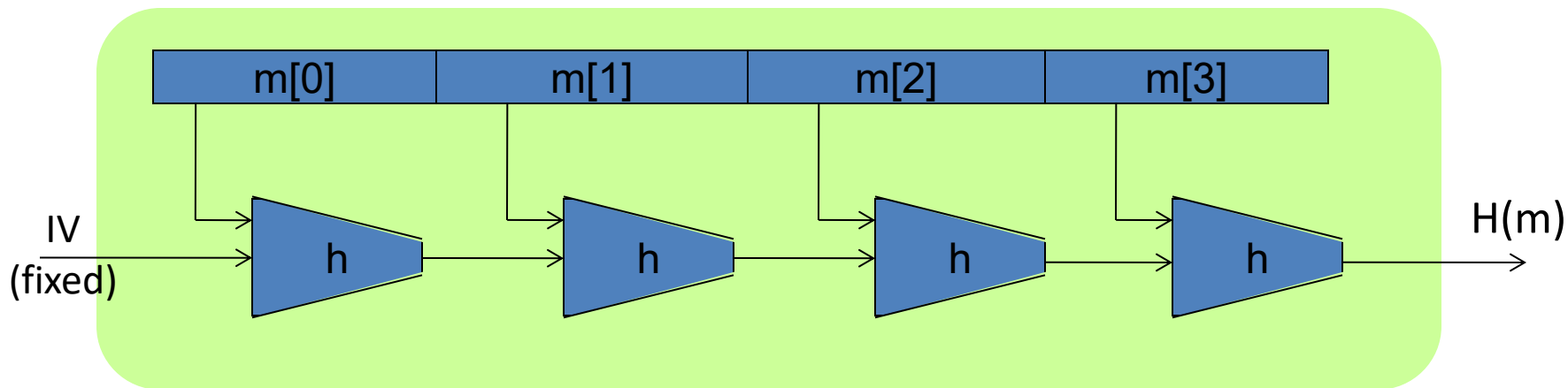
example: SHA-256 ; output is 256 bits

Building a MAC out of a hash function:

– Standardized method: HMAC

$$S(k, \text{msg}) = H(k \oplus \text{opad} \parallel H(k \oplus \text{ipad} \parallel \text{msg}))$$

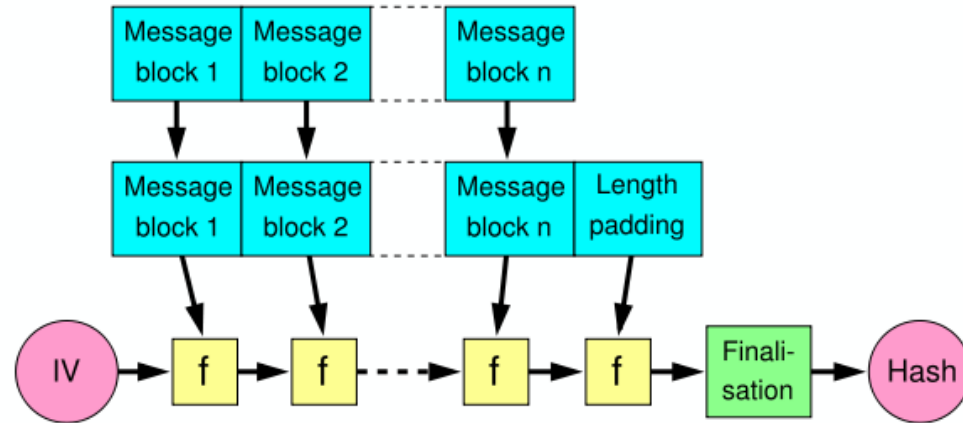
SHA-256: Merkle-Damgard



- $h(t, m[i])$: compression function
- Thm 1: if h is collision resistant then so is H
- “Thm 2”: if h is a “PRF” then HMAC is a secure MAC

Merkle-Damgård Construction for Hash Functions

- Message is divided into fixed-size blocks and padded
- Uses a compression function f , which takes a chaining variable (of size of hash output) and a message block, and outputs the next chaining variable
- Final chaining variable is the hash value



$$M = m_1 m_2 \dots m_n; C_0 = IV, C_{i+1} = f(C_i, m_i); H(M) = C_n$$

Combining MAC and ENC (Auth. Enc.)

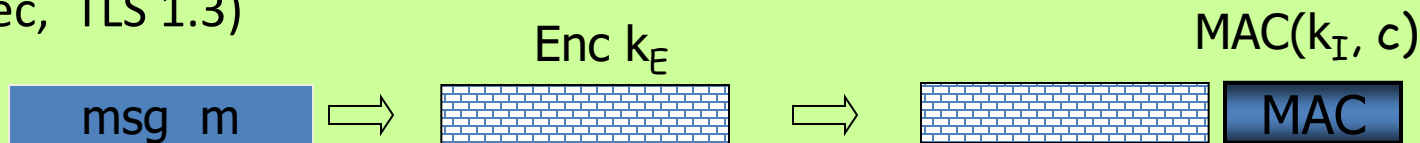
Encryption key k_E . MAC key = k_I

Option 1: (SSL)



Option 2: (IPsec, TLS 1.3)

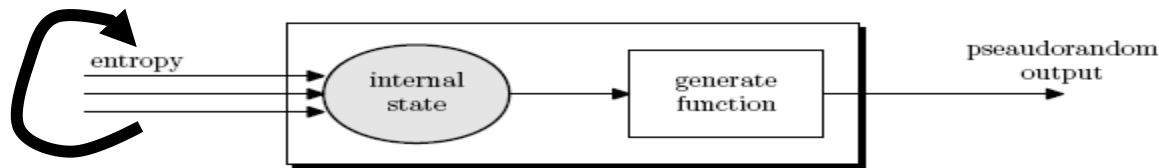
**always
correct**



Option 3: (SSH)



Generating Randomness (e.g. keys, nonces)



Pseudo random generators in practice: (e.g. `/dev/random`)

- Continuously add entropy to internal state
- Entropy sources:
 - Hardware RNG: Intel **RdRand** inst. (Ivy Bridge). 3Gb/sec.
 - Timing: hardware interrupts (keyboard, mouse)

Security Requirements for Cryptographic Hash Functions

Given a function $h:X \rightarrow Y$, then we say that h is:

- **preimage resistant (one-way):**

if given $y \in Y$ it is computationally infeasible to find a value $x \in X$ s.t. $h(x) = y$

- **2-nd preimage resistant (weak collision resistant):**

if given $x \in X$ it is computationally infeasible to find a value $x' \in X$, s.t. $x' \neq x$ and $h(x') = h(x)$

- **collision resistant (strong collision resistant):**

if it is computationally infeasible to find two distinct values $x', x \in X$, s.t. $h(x') = h(x)$

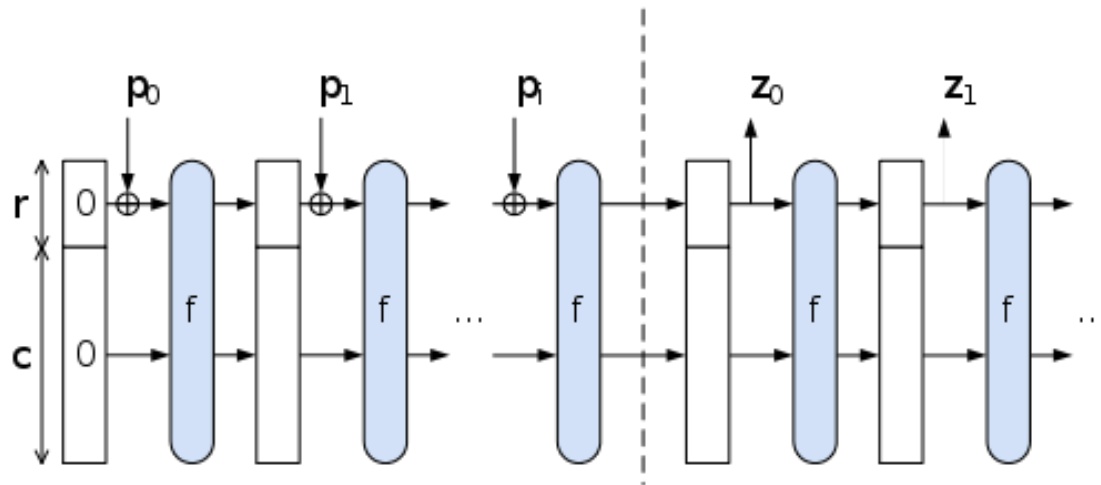
Well Known Hash Functions

- MD5
 - output 128 bits
 - collision resistance completely broken by researchers in China in 2004
- SHA1
 - output 160 bits
 - no collision found yet, but method exist to find collisions in less than 2^{80}
 - considered insecure for collision resistance
 - one-wayness still holds
- SHA2 (SHA-224, SHA-256, SHA-384, SHA-512)
 - outputs 224, 256, 384, and 512 bits, respectively
 - No real security concerns yet

NIST SHA-3 Competition

- NIST is having an ongoing competition for SHA-3, the next generation of standard hash algorithms
- 2007: Request for submissions of new hash functions
- 2008: Submissions deadline. Received 64 entries. Announced first-round selections of 51 candidates.
- 2009: After First SHA-3 candidate conference in Feb, announced 14 Second Round Candidates in July.
- 2010: After one year public review of the algorithms, hold second SHA-3 candidate conference in Aug. Announced 5 Third-round candidates in Dec.
- 2011: Public comment for final round
- 2012: October 2, NIST selected SHA3
 - Keccak (pronounced “catch-ack”) created by Guido Bertoni, Joan Daemen and Gilles Van Assche, Michaël Peeters

The Sponge Construction: Used by SHA-3



- Each round, the next r bits of message is XOR'ed into the first r bits of the state, and a function f is applied to the state.
- After message is consumed, output r bits of each round as the hash output; continue applying f to get new states
- SHA-3 uses 1600 bits for state size

Choosing the length of Hash outputs

- The Weakest Link Principle:
 - A system is only as secure as its weakest link.
- Hence all links in a system should have similar levels of security.
- Because of the birthday attack, the length of hash outputs in general should double the key length of block ciphers
 - SHA-224 matches the 112-bit strength of triple-DES (encryption 3 times using DES)
 - SHA-256, SHA-384, SHA-512 match the new key lengths (128,192,256) in AES

Limitation of Using Hash Functions for Authentication

- Require an authentic channel to transmit the hash of a message
 - Without such a channel, it is insecure, because anyone can compute the hash value of any message, as the hash function is public
 - Such a channel may not always exist
- How to address this?
 - use more than one hash functions
 - use a key to select which one to use

Encryption and compression: oil and vinegar

HTTP: uses compression to reduce bandwidth

Option 1: first encrypt and then compress

- Does not work ... ciphertext looks like a random string

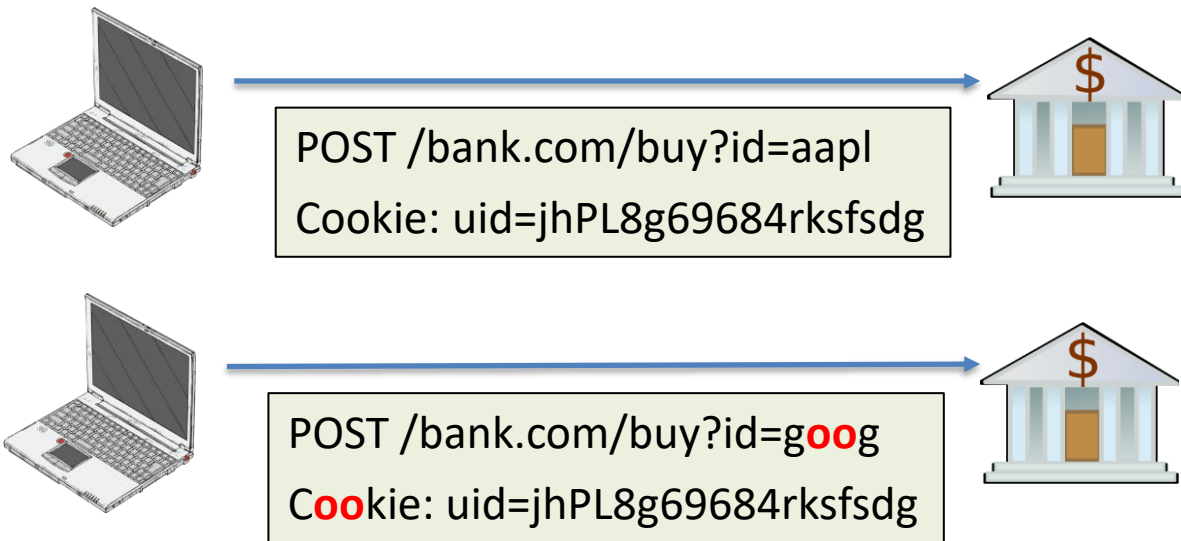
Option 2: first compress and then encrypt

- Used in many Internet protocols (TLS, HTTP, QUIC, ...)
- Trouble ...

Trouble ...

[Kelsey'02]

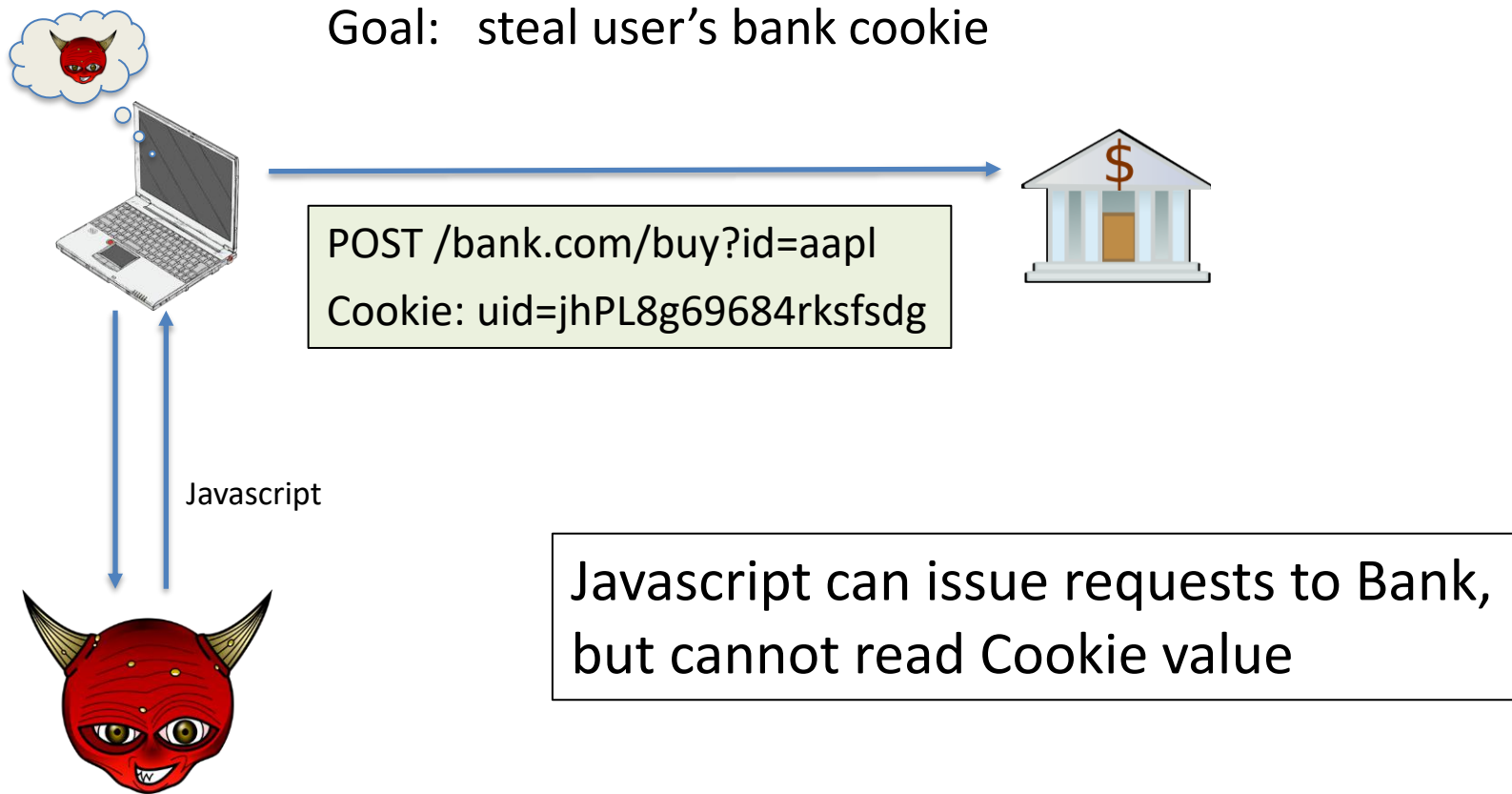
Compress-then-encrypt reveals information:



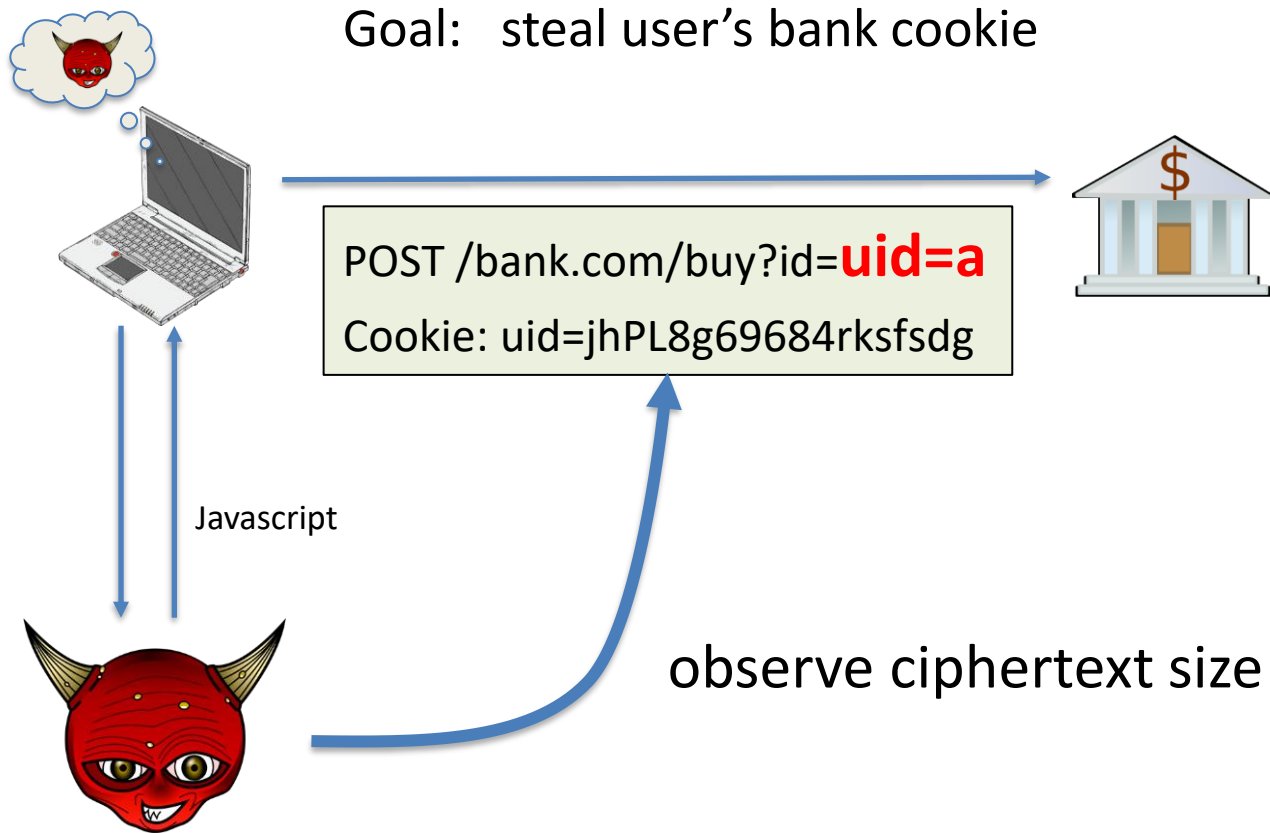
Second message compresses better than first:

network observer can distinguish the two messages!

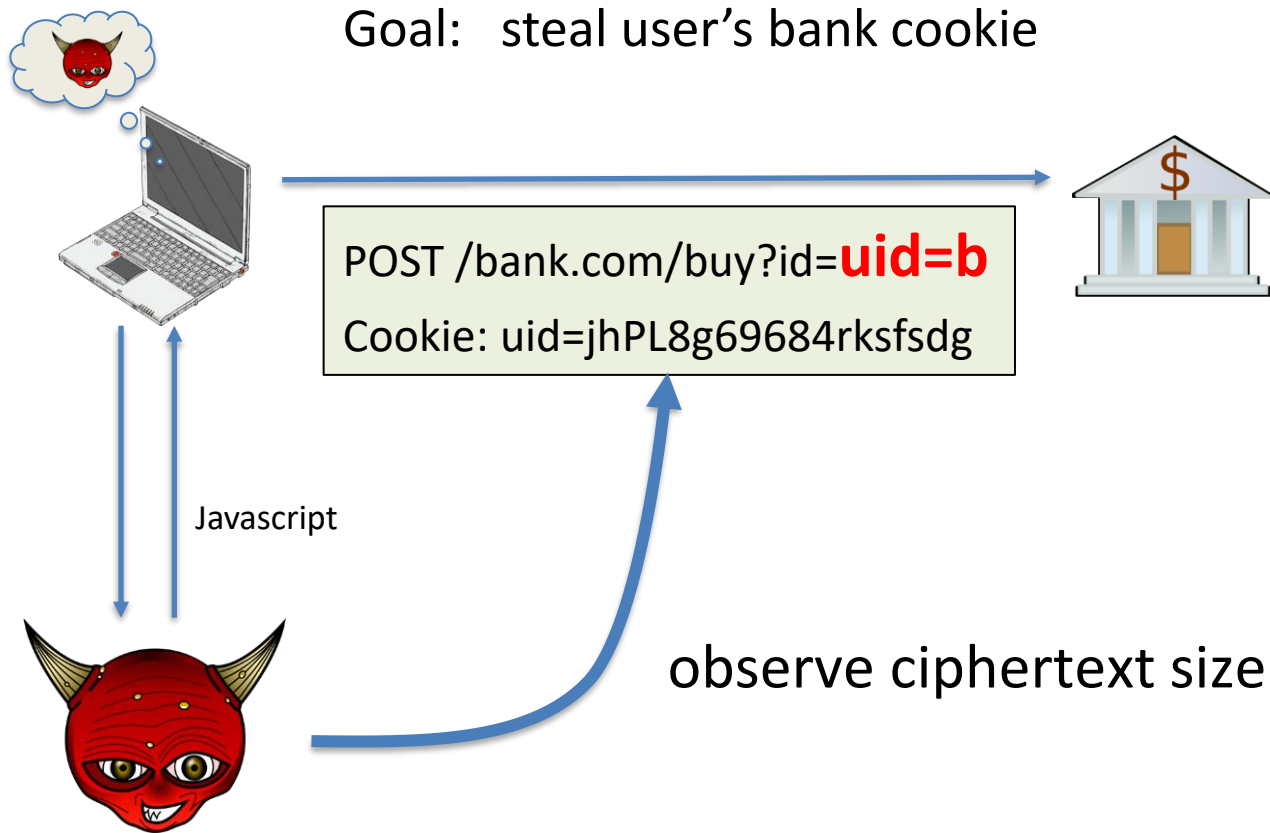
Even worse: the CRIME attack [RD'2012]



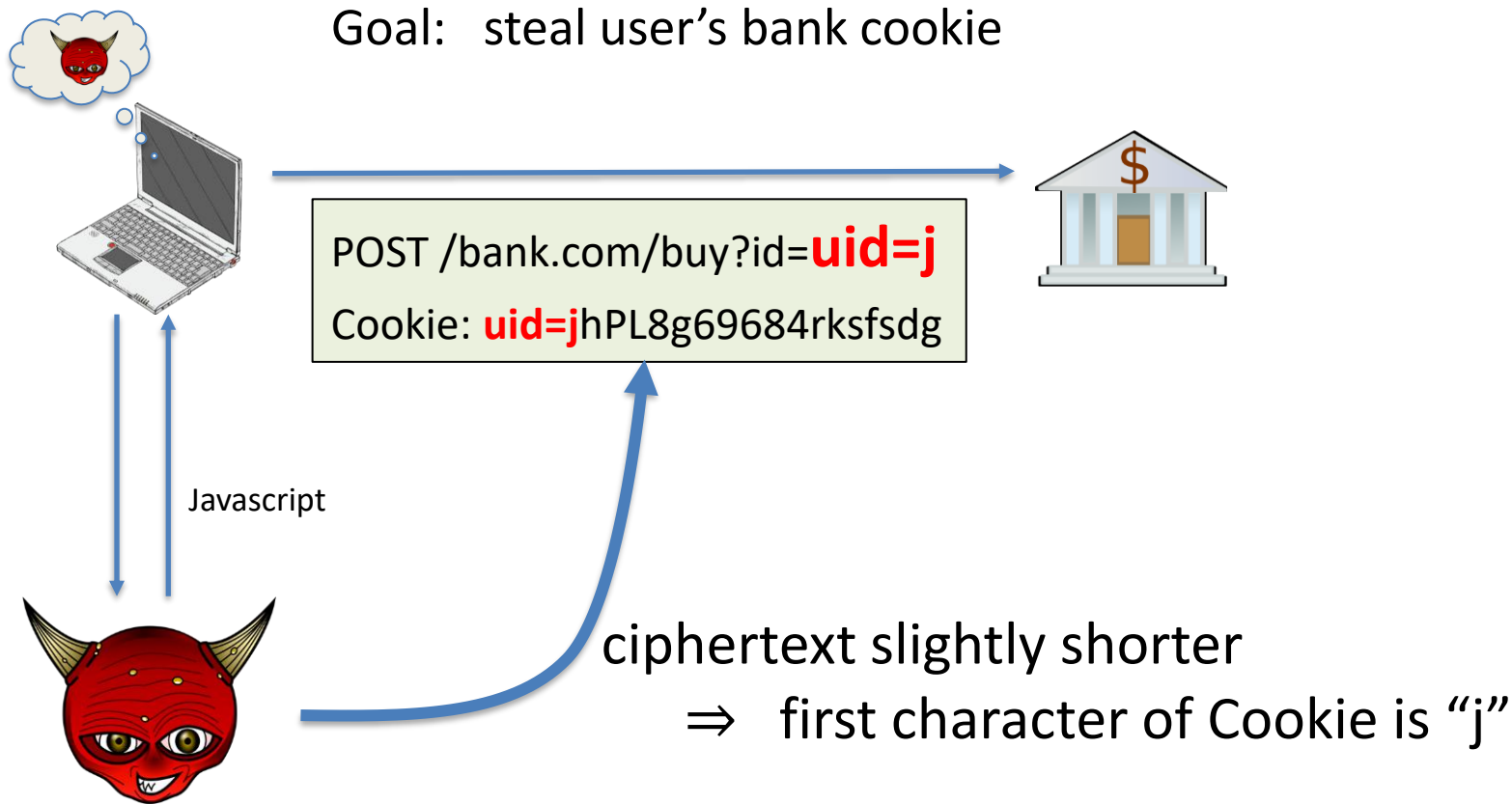
Even worse: the CRIME attack [RD'2012]



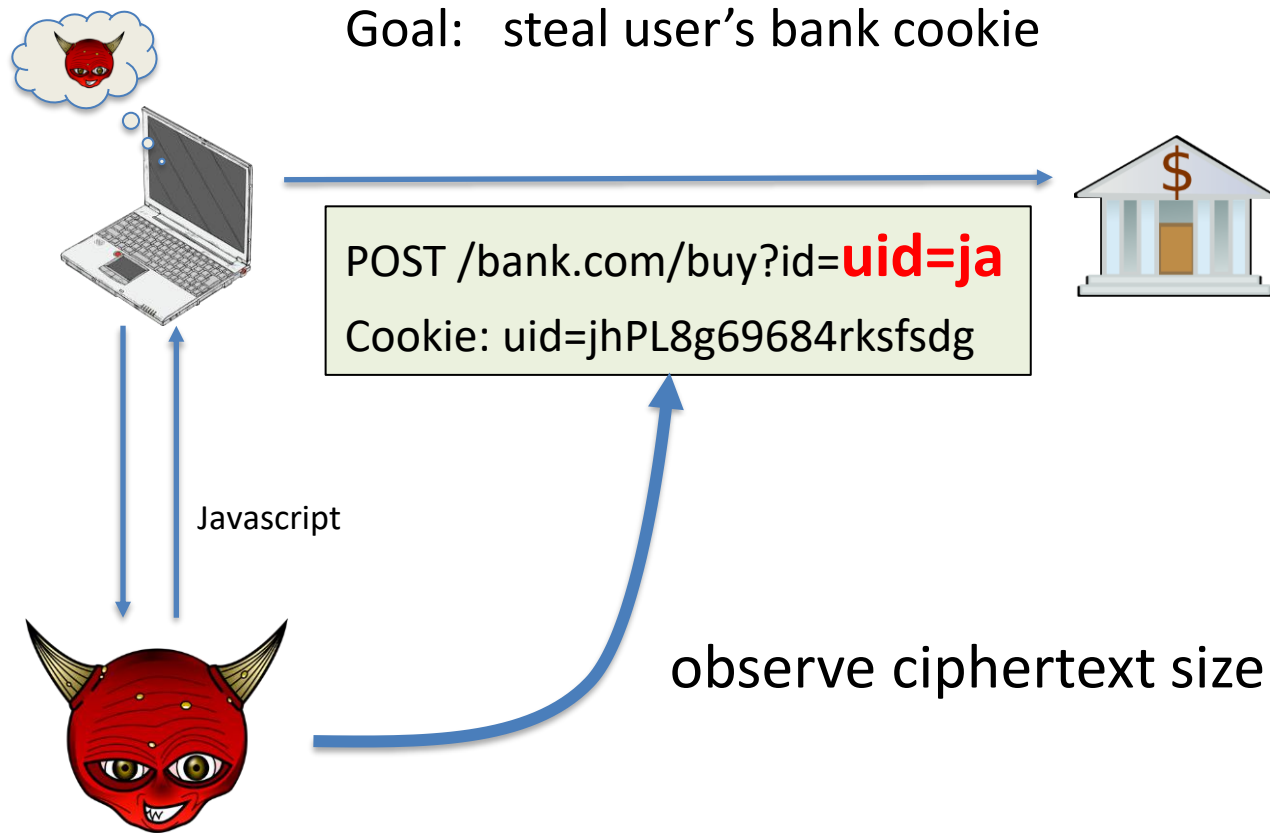
Even worse: the CRIME attack [RD'2012]



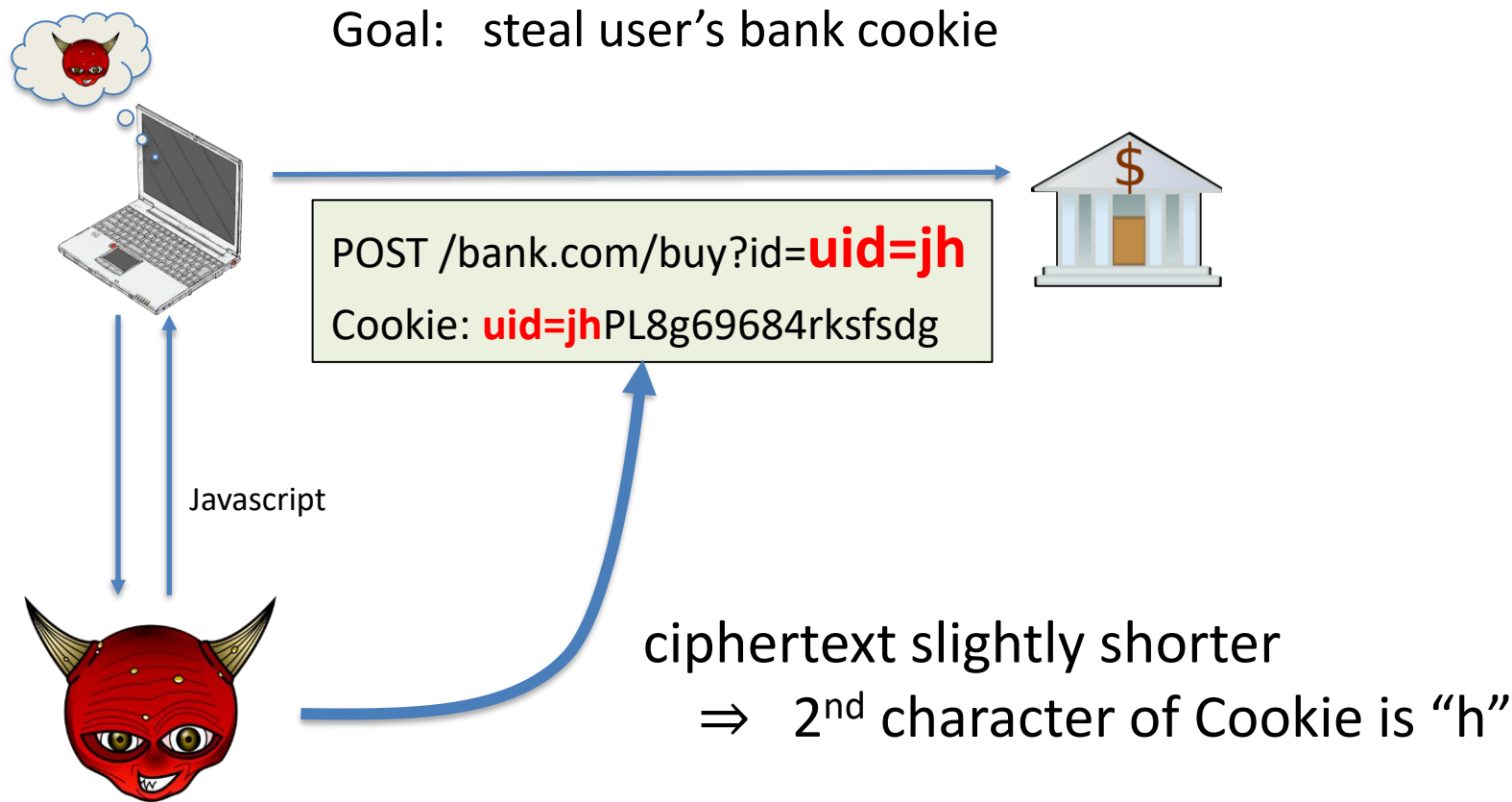
Even worse: the CRIME attack [RD'2012]



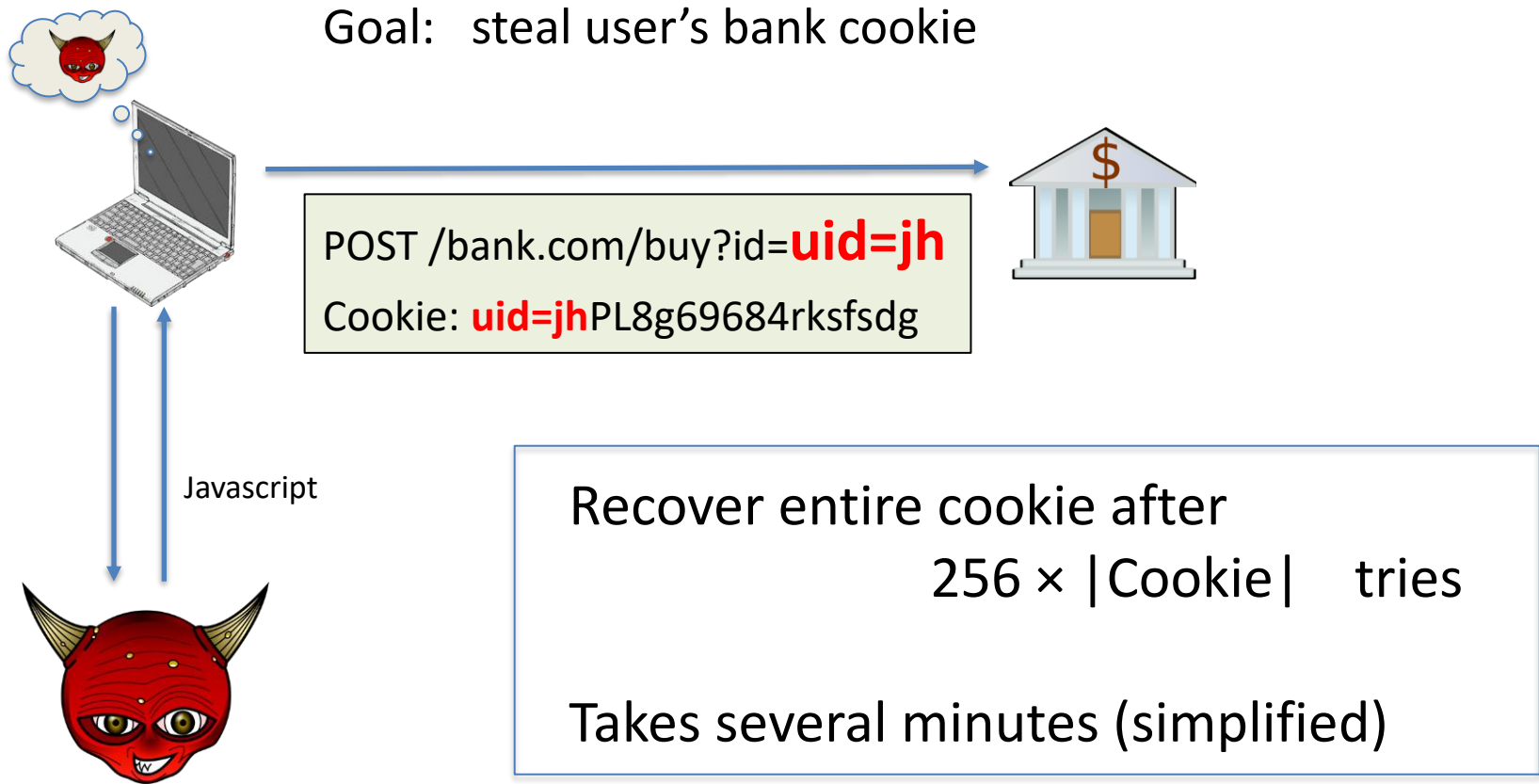
Even worse: the CRIME attack [RD'2012]



Even worse: the CRIME attack [RD'2012]

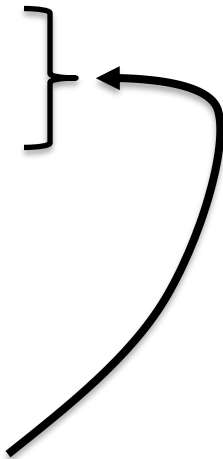


Even worse: the CRIME attack [RD'2012]



What to do?

- Disable compression 🙄
- Use a different compression context for parts under Javascript control and parts that are not
- Change secret (Cookie) after every request



Does not eliminate inherent leakage due to compression

(2) Digital signatures

Goal: bind document to author

- Problem: attacker can copy Alice's sig from one doc to another

Main idea: make signature depend on document

Example: signatures from a trapdoor permutation (e.g. RSA)

$$\text{sign}(sk, m) := F^{-1}(sk, H(m))$$

$$\text{verify}(pk, m, \text{sig}) := \text{accept if } F(pk, \text{sig}) = H(m)$$

- Anyone who has **pk** can verify a (msg, signature) pair

Goal: bind document to author

- Problem: attacker can copy Alice's sig from one doc to another

Main idea: make signature depend on document

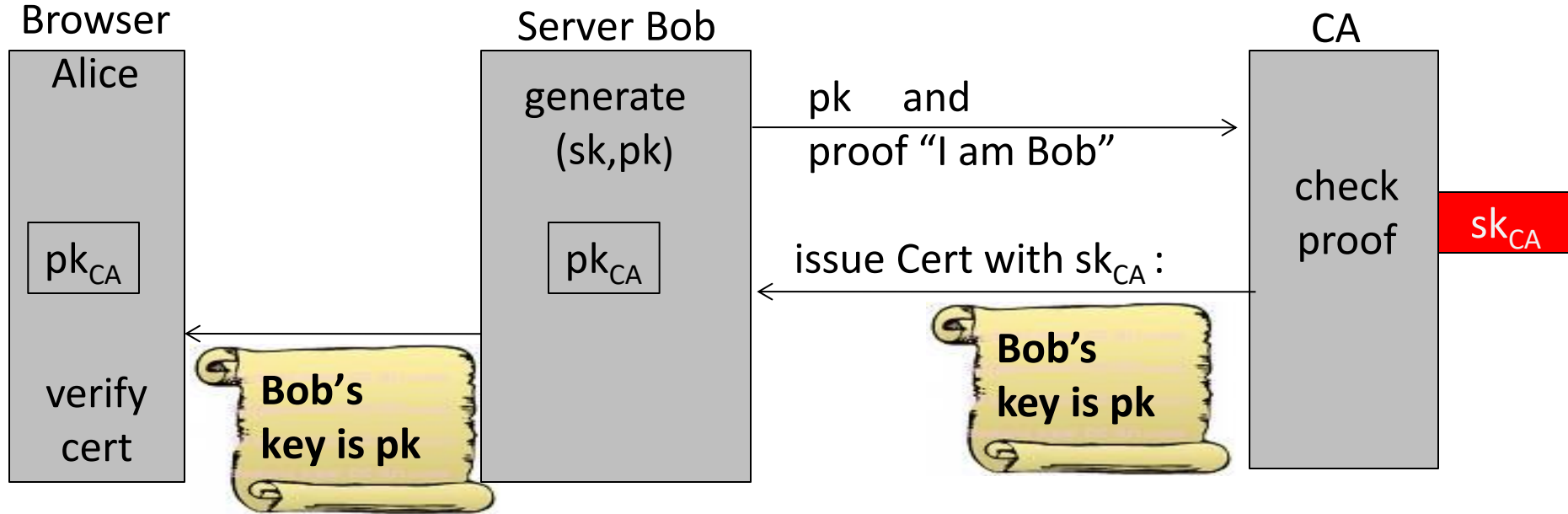
Example: signatures from a trapdoor permutation (e.g. RSA)

$$\text{sign}(\text{sk}, m) := F^{-1}(\text{sk}, H(m))$$

$$\text{verify}(\text{pk}, m, \text{sig}) := \text{accept if } F(\text{pk}, \text{sig}) = H(m)$$

Certificates: bind Bob's ID to a PK

How does Alice (browser) obtain Bob's public key pk_{Bob} ?



Bob uses Cert for an extended period (e.g. one year)



mail.google.com

Issued by: GTS CA 1C3

Expires: Sunday, June 19, 2022 at 7:26:20 PM Pacific Daylight Time

▼ Details

Subject Name
Country US
State/Province California
Locality Mountain View
Organization Google Inc
Common Name mail.google.com

Issuer Name
Country US
Organization Google Trust Services
Common Name Google Internet Authority G3

Serial Number 3495829599616174946
Version 3
Signature Algorithm SHA-256 with RSA

Public Key Info
Algorithm Elliptic Curve Public Key (1.2.840.10045.2.1)
Parameters Elliptic Curve secp256r1 (1.2.840.10045.3.1.7)
Public Key 65 bytes : 04 D5 63 FC 4D F9 4E 91 ...
Key Size 256 bits
Key Usage Encrypt, Verify, Derive
Signature 256 bytes : 3F FE 04 7B BE B0 32 1D ...

Sample certificate:

Signature schemes used in the real world

RSA signature scheme:

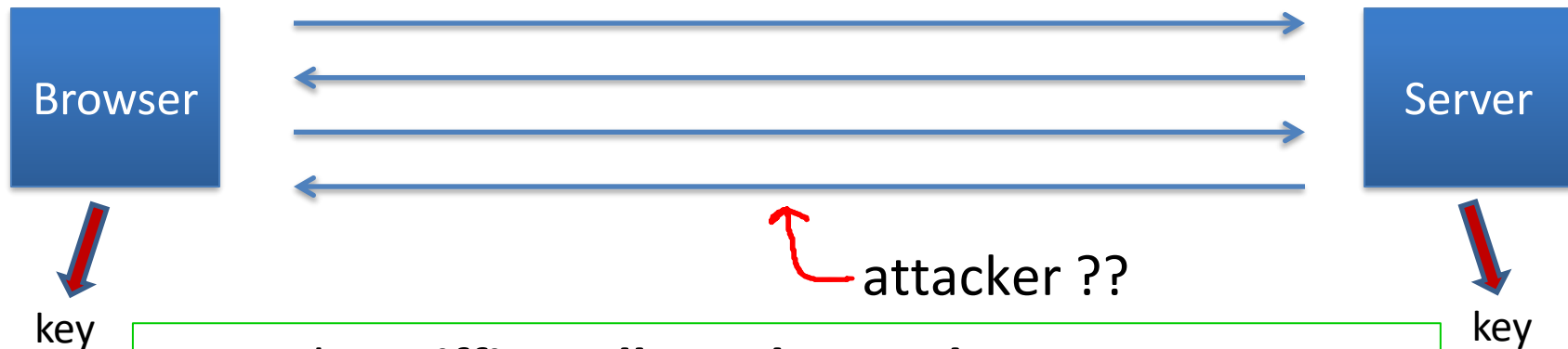
- Fast to verify, but signatures are long
- Often used in certificates

ECDSA, Schnorr, BLS signature schemes:

- Faster to generate signature and more compact than RSA
- Used everywhere, other than web certificates

(3) Key exchange

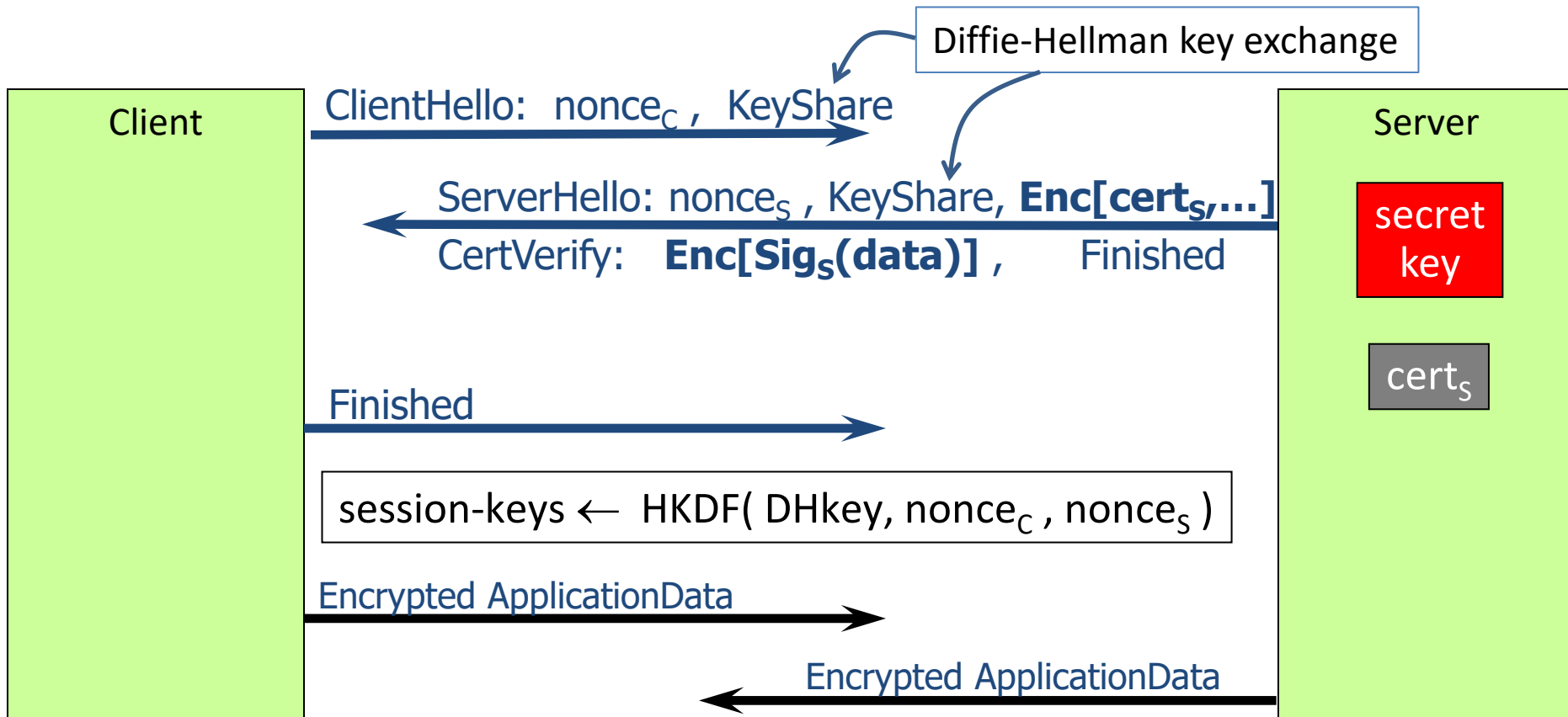
Goal: Browser and Server want a shared secret, unknown to attacker



Example: **Diffie-Hellman key exchange.**

- Only secure against eavesdropping
- TLS 1.3: enhances Diffie-Hellman key exchange
⇒ security against an active attacker

TLS 1.3 session setup (simplified)



Properties

■ Connection - secure (strong TLS 1.3)

The connection to this site is encrypted and authenticated using TLS 1.3 (a strong protocol), X25519 (a strong key exchange), and AES_128_GCM (a strong cipher).

Gmail

Nonces: prevent replay of an old session

Forward secrecy: server compromise does not expose old sessions

Some identity protection: certificates are sent encrypted

One sided authentication:

- Browser identifies server using server-cert
- TLS has support for mutual authentication
 - requires a client pk/sk and client-cert

Summary: crypto concepts

Symmetric cryptography:

Authenticated Encryption (AE) and message integrity

Public-key cryptography:

Public-key encryption, digital signatures, key exchange

Certificates: bind a public key to an identity using a CA

- Used in TLS to identify server (and possibly client)

Modern crypto: goes far beyond basic encryption and signatures